

Final project – Report

MCU and HW-Accelerators architecture and design

נועם קורין פרי – 208138586

רועי אגושביץ – 322720665



תוכן עניינים

1. הקדמה – עמוד 2
2. הסבר על המערכת – עמוד 3
3. תיאור קבצי ה־HDL – עמוד 4
4. תצוגות RTL של המערכת – עמוד 8
5. אימות פונקציונלי באמצעות ModelSim – עמוד 12
 - 5.1 Test1 – Pushbutton Interrupts and Divider – עמוד 12
 - 5.2 Test2 – Basic Timer Periodic Interrupt – עמוד 13
 - 5.3 Test3 – Reprogramming the Basic Timer – עמוד 13
 - 5.4 Test4 – PWM / Output Compare – עמוד 14
 - 5.5 Test4 – Input Capture and Divider – עמוד 15
6. אימות חומרתי באמצעות SignalTap – עמוד 16
 - 6.1 Test1 – KEY1 Interrupt – עמוד 16
 - 6.2 Test2 – Basic Timer Interrupt – עמוד 17
 - 6.3 Test3 – Basic Timer Interrupt – עמוד 17
 - 6.4 Test4 – Integrated Hardware Validation – עמוד 18
 - 6.4.1 Basic Timer Interrupt and CPU Response – עמוד 18
 - 6.4.2 KEY1 Interrupt – עמוד 18
 - 6.4.3 Divider Accelerator – עמוד 19
 - 6.4.4 PWM – Output Compare – עמוד 20
7. אפיון ה־PPA של המערכת – עמוד 22
 - 7.1 Area – ניצול משאבי FPGA – עמוד 22
 - 7.2 Performance – תדר מקסימלי והנתיב הקריטי – עמוד 24
 - 7.3 Power – צריכת הספק – עמוד 27
8. מסקנות – עמוד 29

1. הקדמה

מטרת הפרויקט היא לתכנן ולממש מערכת MCU המבוססת על ליבת RV32IM בארכיטקטורת Single-Cycle, ולשלב סביבה מלאה הכוללת זיכרונות, רכיבי קלט/פלט, פסיקות ומאיץ חומרה. בדומה למעבד ממעבדה 5, הליבה מבוססת על ארכיטקטורת Harvard ובה זיכרון פקודות ITCM וזיכרון נתונים DTCM, אך בפרויקט זה הורחבה המערכת כך שתתפקד כ-MCU מלא באמצעות Memory Mapped I/O ורכיבים פריפריאליים נוספים.

במסגרת הפרויקט מומשו ממשקי GPIO עבור הלדים, תצוגות ה-7-Segment והמתגים, ממשק ללחצנים KEY1–KEY3 עם תמיכה בפסיקות, Basic Timer הכולל Compare Mode, Output Compare ליצירת PWM ו-Input Capture, וכן Interrupt Controller האחראי על זיהוי מקור הפסיקה, קביעת עדיפויות והעברת ערך ה-TYPE למעבד. בנוסף שולב מאיץ חלוקה רב-מחזורי עבור פקודות DIV ו-REM, הפועל בדומיין שעון מהיר יותר ודורש מנגנון סנכרון בין תחומי השעון.

תהליך העבודה כלל כתיבת קוד VHDL ואינטגרציה של כלל רכיבי המערכת, אימות פונקציונלי ב-ModelSim באמצעות אפליקציות ה-Benchmark, אימות על גבי כרטיס ה-FPGA באמצעות SignalTap, ולבסוף אפיון PPA של המערכת ב-Quartus מבחינת שטח, ביצועים וצריכת הספק.

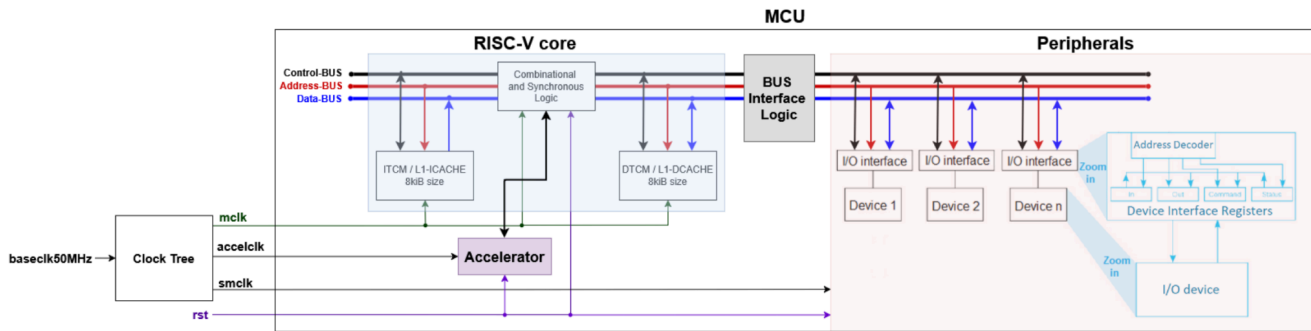
2. הסבר על המערכת

המערכת שתוכננה בפרויקט היא MCU המבוסס על ליבת RV32IM Single-Cycle. הליבה מחוברת לשני זיכרונות נפרדים בהתאם לארכיטקטורת Harvard: זיכרון ITCM עבור קוד התוכנית וזיכרון DTCM עבור נתונים. בנוסף לזיכרונות, המעבד מחובר למרחב כתובות של רכיבי Memory Mapped I/O, כך שפקודות load ו-store משמשות גם לגישה לרכיבים הפריפריאליים של המערכת. לפי חלוקת מרחב הכתובות, כתובות הזיכרון הנמוכות משמשות את ה-DTCM, בעוד הכתובות החל מ-0x2000 משמשות את רכיבי הקלט/פלט והבקרה.

רכיבי ה-GPIO כוללים ממשקים ל-LED, לתצוגות HEX0–HEX5 ולמתגים SW. עבור רכיבים בעלי יכולת פסיקה נוספו ממשק ללחצנים KEY1–KEY3, ה-Basic Timer ובקר הפסיקות. ה-Basic Timer כולל רגיסטרי בקרה והשוואה ומאפשר יצירת פסיקות מחזוריות, אות PWM ומדידת זמן באמצעות Input Capture. בקר הפסיקות אוסף את דגלי הפסיקה, בודק אילו מקורות מאופשרים, קובע את המקור בעל העדיפות הגבוהה ביותר ומייצר את ערך ה-TYPE המתאים לפי טבלת וקטורי הפסיקות.

בנוסף לרכיבים הפריפריאליים, משולב במערכת מאיץ חלוקה רב-מחזורי. המאיץ מבצע פעולות חלוקה ושארית במשך 32 מחזורי DIVCLK, בעוד המעבד עצמו פועל על MCLK. בזמן פעולת המאיץ המעבד ממתין לסיום החישוב, ולאחר מכן התוצאה מוחזרת לנתיב ה-Writeback. מכיוון שמדובר בשני תחומי שעות שונים, שולב גם מנגנון CDC לצורך סנכרון האותות ביניהם.

במימוש הסופי נעשה שימוש בשלושה שעונים: MCLK עבור המעבד, SMCLK עבור ה-Basic Timer ו-DIVCLK עבור מאיץ החלוקה. מבנה זה מאפשר להפעיל את המאיץ בתדר גבוה יותר מהליבה, תוך שמירה על פעולה מסודרת של שאר רכיבי ה-MCU.



איור 1 – מבנה ה־Top Level של מערכת ה־RV32IM-based MCU.

3. תיאור קבצי ה־HDL

להלן תיאור קצר של קבצי ה־HDL המרכיבים את המימוש הסופי של ה־MCU. קבצי ה־DUT כוללים את ליבת ה־RV32IM, זיכרונות ויחידות הבקרה, רכיבי ה־MMIO, בקר הפסיקות, ה־Basic Timer ומאיץ החלוקה. רשימת הקבצים תואמת למימוש שנמצא ב־DUT/RV32IMscMCU.

קובץ	תיאור
RV32IMscMCU.vhd	קובץ ה־Top Level של המערכת. מחבר באופן Structural בין ליבת המעבד, ה־DTCM, ה־Interconnect, הרכיבים הפריפריאליים ומערכת השעונים, ומחבר את כניסות ויציאות המערכת לכרטיס ה־FPGA.
RV32I_CORE.vhd	קובץ ה־Top Level של ליבת ה־RV32IM Single-Cycle. מחבר את יחידות ה־Fetch, Decode, Control ו־Execute, וכן את מאיץ ה־Divider ואת מנגנון הטיפול בפסיקות.

IFETCH.VHD	אחראי על ה־Program Counter ועל שליפת הפקודה מזיכרון ה־ITCM. בנוסף בוחר את כתובת הפקודה הבאה בהתאם לרצף רגיל, Branch או Jump.
IDECODE.VHD	מבצע את שלב פענוח הפקודה, כולל קריאת האופרנדים מה־Register File, כתיבה חזרה לרגיסטרים ויצירת ערכי ה־Immediate השונים.
CONTROL.VHD	יחידת הבקרה הקומבינטורית של המעבד. מפענחת את הפקודה ומייצרת את אותות הבקרה עבור ה־ALU, הזיכרון, Branch/Jump ופקודות ה־M Extension כגון MUL, DIV ו־REM.
EXECUTE.VHD	מבצע את הפעולות האריתמטיות והלוגיות של ה־ALU, חישובי כתובות וזיהוי תנאי Branch. בנוסף משולב בו נתיב הביצוע עבור פעולות הכפל.
DMEMORY.VHD	מממש את זיכרון הנתונים DTCM, כולל פעולות הקריאה והכתיבה של המעבד.
Multiplier16.vhd	מממש את יחידת הכפל של פקודת MUL, המבוססת על הכפלת שני האופרנדים והפקת תוצאת הכפל לנתיב הביצוע.
PLL.vhd	מממש את ה־PLL המשמש להפקת שעון העבודה של המערכת משעון ה־50 MHz של הכרטיס. במימוש הסופי

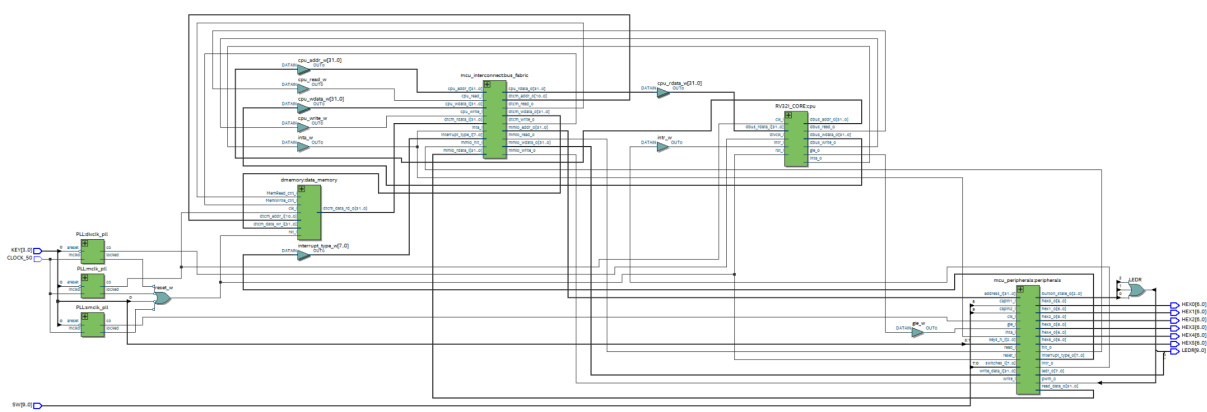
ליבת המעבד פועלת ב-20 MHz, בעוד מאיץ החלוקה משתמש ב-50 MHz.	
מממש את ה-Basic Timer ואת רגיסטרי הבקרה שלו. הטיימר תומך ב-Compare Mode ליצירת פסיקות מחזוריות, Output Compare ליצירת PWM ו-Input Capture לשמירת ערך המונה בעת אירוע חיצוני.	basic_timer.vhd
מממש את בקר הפסיקות של המערכת. שומר את רגיסטרי IE ו-IFG, קובע את מקור הפסיקה בעל העדיפות הגבוהה ביותר, מפיק את ערך ה-TYPE ומייצר את אות INTR למעבד.	interrupt_controller.vhd
מממש את ממשק KEY1–KEY3. מאפשר קריאה של מצב הלחצנים ומייצר אירוע פסיקה בעת שחרור לחצן, בהתאם למעבר מ-0 ל-1 של אות ה-KEY.	pushbutton_unit.vhd
מממש את רכיבי ה-GPIO הממופים לזיכרון: LEDR, תצוגות HEX0–HEX5 והמתגים SW, כולל קריאה וכתיבה לפי כתובות ה-MMIO.	gpio_peripheral.vhd
מרכז את כלל הרכיבים הפריפריאליים של ה-MCU ומחבר ביניהם לבין ממשק ה-MMIO. כולל את ה-GPIO, הלחצנים, ה-Basic Timer ובקר הפסיקות.	mcu_peripherals.vhd
מממש את לוגיקת ה-Bus Interface. בודק את הכתובת שמפיק המעבד ומנתב כל פעולת Load/Store אל	mcu_interconnect.vhd

ה־DTCM או אל מרחב ה־MMIO, ומחזיר למעבד את הנתונים מהיעד המתאים.	
מכיל את קבועי מפת הזיכרון של ה־MCU, ובהם תחומי ה־DTCM וה־MMIO והכתובות של רכיבי ה־GPIO, הטיימר ובקר הפסיקות.	mcu_memory_map_pkg.vhd
מממש את ליבת המחלק הבלתי־מסומן. פעולת החלוקה מתבצעת באלגוריתם רב־מחזורי במשך 32 מחזורי שעות ומפיקה מנה ושארית.	divider_unsigned.vhd
עוטף את המחלק ומחבר אותו לליבת המעבד. הקובץ מטפל בפקודות DIV/REM, באותות busy/done ובמעבר בין תחום שעות ה־MCLK לתחום ה־DIVCLK באמצעות מנגנון CDC.	divider_accelerator.vhd
מכיל קבועים משותפים לליבת המעבד, כגון Opcodes, מסכות לפענוח פקודות וקבועים המשמשים את יחידות ה־Control וה־Datapath.	const_package.vhd
מכיל פרמטרים גלובליים של התכן, כגון גדלי הזיכרונות ורוחבי הכתובות, ומשמש להגדרת תצורת הקומפילציה של המערכת.	cond_compilation_package.vhd
מכיל הצהרות Components ו־Generics משותפות המשמשות לחיבור מבני של יחידות המעבד והמערכת.	aux_package.vhd

4. תצוגות RTL של המערכת

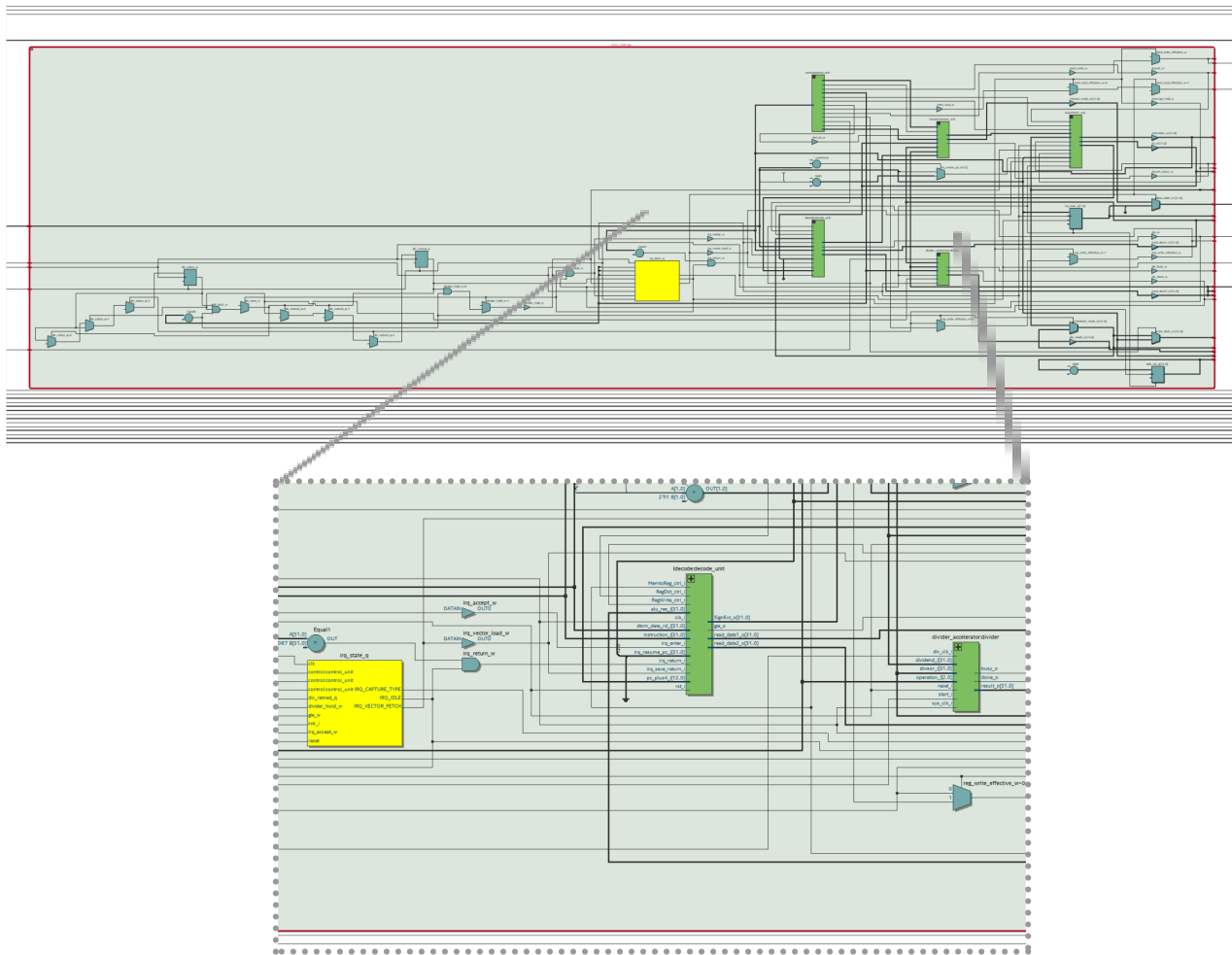
לאחר השלמת הסינתזה ב־Quartus נבדק מבנה המערכת באמצעות ה־RTL Viewer. מטרת הבדיקה היא לוודא שהמודולים המרכזיים של המערכת חוברו כפי שתוכנן, ולראות את ההיררכיה והקשרים בין ליבת המעבד, הזיכרונות, הרכיבים הפריפריאליים ומאיץ החלוקה. דרישה להצגת תוצאות ה־RTL Viewer מופיעה במפורש בסעיף 10 של הוראות הפרויקט.

בתצוגת ה־Top Level ניתן לראות את ליבת ה־RV32IM Single-Cycle מחוברת ל־ITCM ול־DTCM, את ממשק ה־MMIO המחבר את המעבד לרכיבי ה־GPIO ולרכיבים בעלי יכולת פסיקה, וכן את מערכת השעונים והחיבור למאיץ החלוקה. המבנה משקף את ארכיטקטורת ה־MCU הכללית שנדרשה בפרויקט.



איור 2 - תצוגת RTL של ה־Top Level של מערכת ה־RV32IM-based MCU.

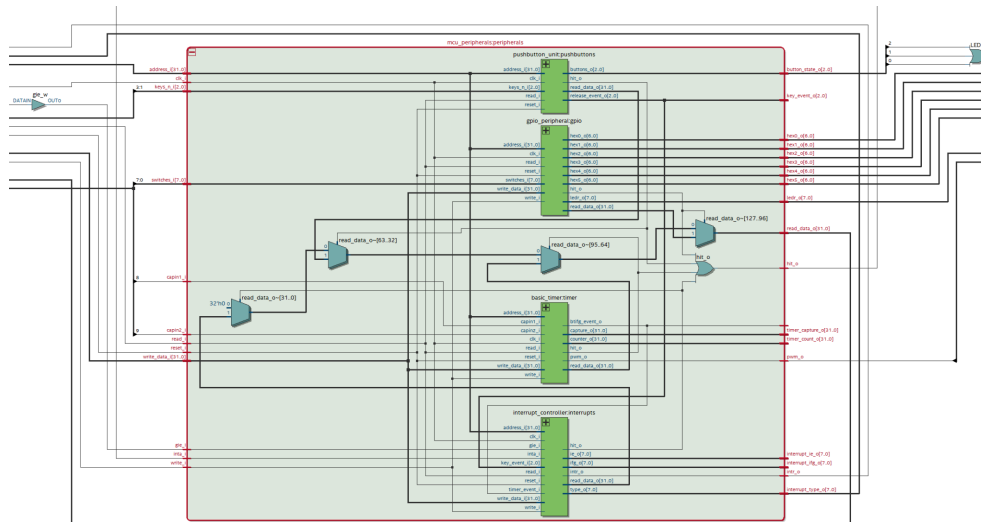
בנוסף נבדקה תצוגת ה־RTL של ליבת המעבד ושל נתיב מאיץ החלוקה. ניתן לראות את יחידות ה־Fetch, Decode, Control ו־Execute של הליבה החד־מחזורית, וכן את החיבור בין האופרנדים של המעבד למאיץ ה־Divider ואת החזרת תוצאת ה־Quotient או ה־Remainder לנתיב ה־Writeback. מבנה זה תואם לארכיטקטורה הנדרשת של ליבת RV32IM עם מאיץ חלוקה רב־מחזורי.



איור 3 - תצוגת RTL של ליבת ה־RV32IM והחיבור למאיץ ה־DIV/REM.

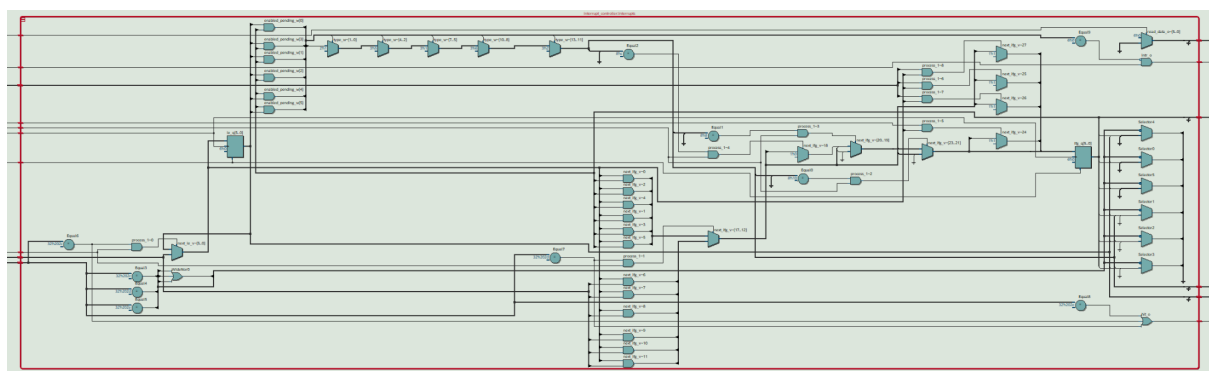
תצוגות נוספות במערכת:

מוצגת גם יחידת `mcu_peripherals`, המרכזת את הרכיבים הפריפריאליים של המערכת. יחידה זו כוללת את ממשקי ה־GPIO, הלחצנים, ה־Basic Timer ובקר הפסיקות, ומחברת אותם לממשק ה־MMIO של ה־MCU. מבנה ה־RTL מראה את חלוקת היחידה לתת־מודולים ואת הקשרים ביניהם.



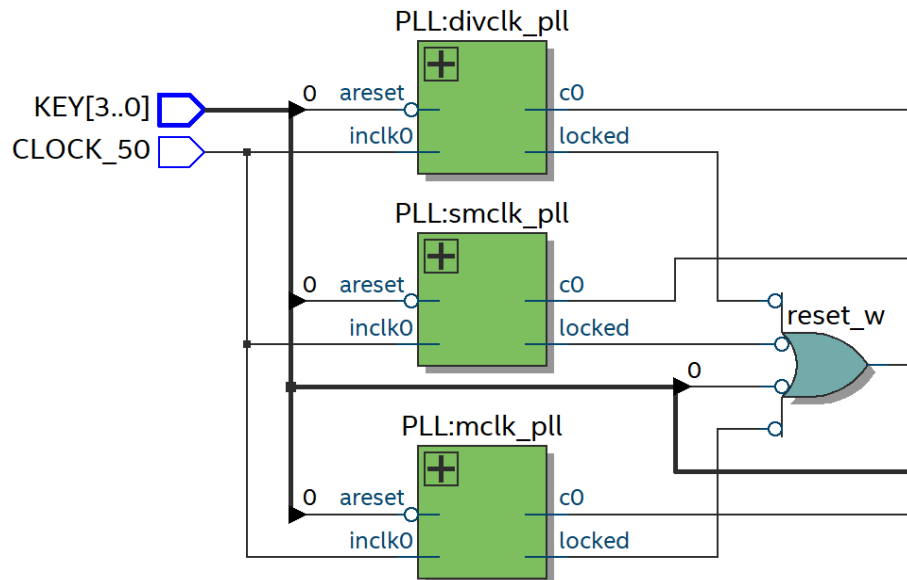
איור 4 – תצוגת RTL של מערך הרכיבים הפריפריאליים של ה-MCU.

בהמשך נבחן גם ה-Interrupt Controller, שהוא אחד הרכיבים המרכזיים שנוספו למערכת לצורך תמיכה בפסיקות. ניתן לזהות בו את הלוגיקה הקשורה לרגיסטרי IE, IFG ו-TYPE, וכן את החיבור למקורות הפסיקה השונים. הבקר אחראי לסינון הפסיקות לפי ביטי ה-Enable, לקביעת מקור הפסיקה בעל העדיפות הגבוהה ביותר ולהעברת בקשת הפסיקה וה-TYPE המתאים אל המעבד.



איור 5 – תצוגת RTL של בקר הפסיקות.

לבסוף נציג גם את מערכת השעונים של ה-MCU. המערכת משתמשת בשלושה אותות שעון נפרדים המופקים מ-CLOCK_50: MCLK עבור ליבת המעבד, SMCLK עבור ה-Basic Timer ו-DIVCLK עבור מאיץ החלוקה. במימוש הסופי MCLK ו-SMCLK פועלים בתדר 20 MHz, בעוד DIVCLK פועל בתדר 50 MHz.



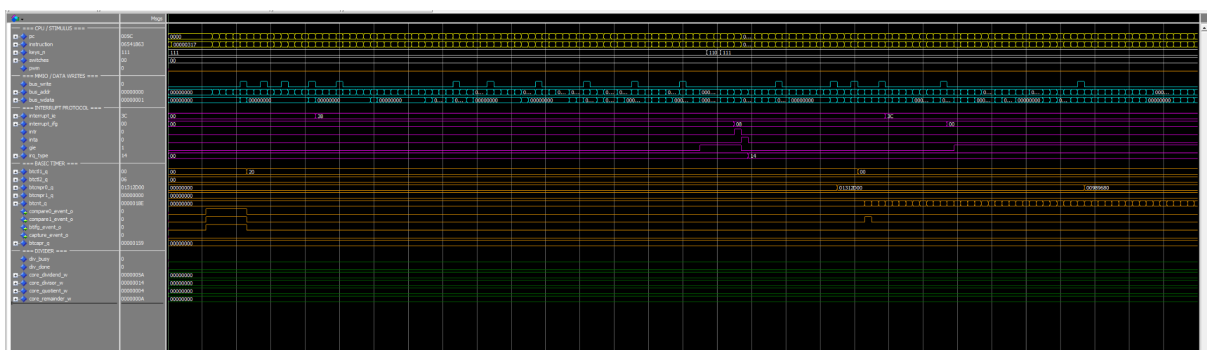
איור 6 – תצוגת RTL של מערכת השעונים וה-PLL של ה-MCU.

5. אימות פונקציונלי באמצעות ModelSim

לאחר השלמת האינטגרציה של רכיבי ה-MCU בוצע אימות פונקציונלי ב-ModelSim באמצעות ארבע אפליקציות ה-Benchmark שניתנו לפרויקט. מטרת הסימולציות הייתה לבדוק לא רק את פעולת כל רכיב בנפרד, אלא גם את שיתוף הפעולה בין ליבת המעבד, ה-Memory Mapped I/O, בקר הפסיקות, ה-Basic Timer ומאיץ החלוקה. במהלך ההרצות נבדקו אותות ה-MMIO, דגלי הפסיקה, אותות INTR ו-INTRA, פעילות ה-Divider וכן רגיסטרי ה-Basic Timer.

Test1 – Pushbutton Interrupts and Divider 5.1

ב-Test1 נבדקה תגובת המערכת לפסיקות הנוצרות מהלחצנים KEY1–KEY3. לחיצות KEY1 ו-KEY2 משמשות להצגת איברים מהמערכים על תצוגות ה-HEX, ואילו KEY3 מפעיל פעולות DIV ו-REM באמצעות מאיץ החלוקה. בתרשים הגלים ניתן לעקוב אחר שרשרת הטיפול בפסיקה החל משינוי מצב ה-KEY, דרך יצירת דגל ה-IFG והעלאת INTR, ועד לאישור הפסיקה על ידי המעבד באמצעות INTRA. בהמשך ניתן לראות גם את פעילות מאיץ ה-Divider ואת החזרת תוצאת החישוב למעבד. לדוגמה, עבור האופרנדים הראשונים $arr1=100$ ו- $arr2=8$ מתקבלים מנה 12 ושארית 4, בהתאם לתוצאה הצפויה. המטרה של הבדיקה היא להראות שמסלול הפסיקה מהלחצן אל המעבד פועל כראוי, ושללאחר הטיפול בפסיקה המעבד מסוגל להפעיל את מאיץ החלוקה ולקבל ממנו תוצאה תקינה.

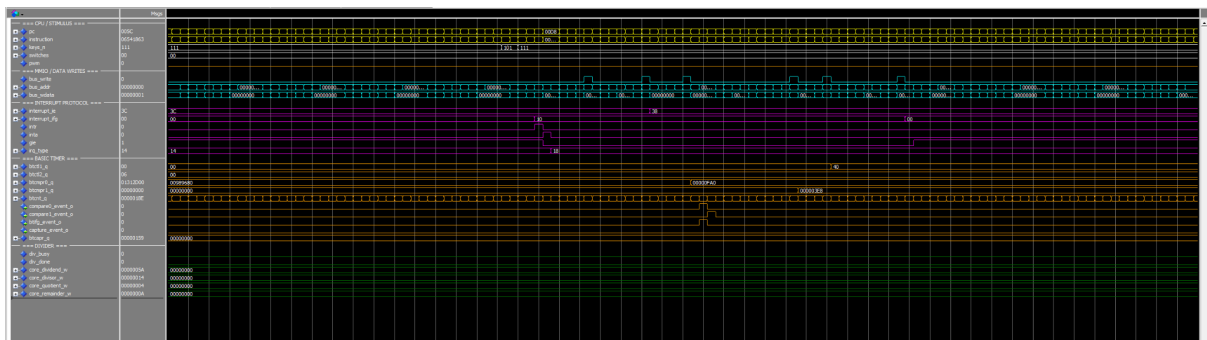


איור 7 – סימולציית ModelSim של Test1 – פסיקות לחצנים ופעולת מאיץ ה-Divider.

Test2 – Basic Timer Periodic Interrupt 5.2

ב־Test2 נבדקה פעולת ה־Basic Timer כמקור לפסיקה מחזורית. בכל אירוע Timer נוצר BTIFG, בקר הפסיקות מזהה את מקור הפסיקה ומעביר למעבד $TYPE=0x10$, בהתאם לטבלת וקטורי הפסיקות. לאחר מכן המעבד מאשר את הפסיקה באמצעות INTA ונכנס לשגרת הטיפול המתאימה.

שגרת הפסיקה מגדילה את הרגיסטר a0, כאשר KEY1–KEY3 קובעים באיזה זוג תצוגות HEX יוצג הערך. בתרשים הגלים ניתן לראות את אירוע הטיימר, עדכון ה־IFG, יצירת INTR, אישור הפסיקה ולבסוף את עדכון הערך. לאחר הפסיקה הראשונה מתקבל $a0=1$, כפי שנדרש באפליקציית ה־Benchmark. בדיקה זו מאמתת את שרשרת הטיפול המלאה בפסיקה פנימית של ה־MCU, החל מה־Basic Timer ועד לביצוע שגרת הפסיקה על ידי המעבד.



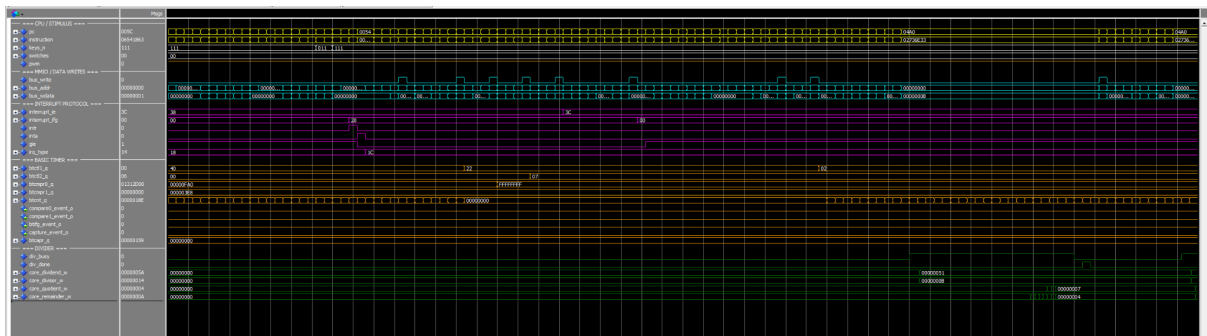
איור 8 – סימולציית ModelSim של Test2 – פסיקה מחזורית של ה־Basic Timer.

Test3 – Reprogramming the Basic Timer 5.3

ב־Test3 נבדקה האפשרות לשנות בזמן ריצה את תקופת הפסיקות של ה־Basic Timer. לחיצות על KEY1, KEY2 ו־KEY3 גורמות לכתיבת ערכים חדשים ל־BTCMPRO, המתאימים לתקופות של $SEC_PERIOD/2$, $SEC_PERIOD/4$ ו־ $SEC_PERIOD/8$. בתרשים הגלים ניתן לראות את כתיבות ה־MMIO לרגיסטר ההשוואה ואת השינוי בהתנהגות הטיימר לאחר התכנות מחדש. בנוסף נבדקה פעולת מנגנון ה־double buffering של רגיסטר ההשוואה: הערך החדש נכתב ל־BTCMPRO ומועבר ל־Latch המשמש בפועל להשוואה מול BTCNT.

כך ששינוי ערך ההשוואה מתבצע בצורה מבוקרת. מנגנון זה תואם למבנה ה-Basic Timer שהוגדר בפרויקט.

הבדיקה מראה כי תוכנת ה-Benchmark מסוגלת לשנות את תקופת הטיימר בזמן ריצה באמצעות כתיבות MMIO, וכי הפסיקות הבאות מתקבלות בהתאם לערך ההשוואה החדש.

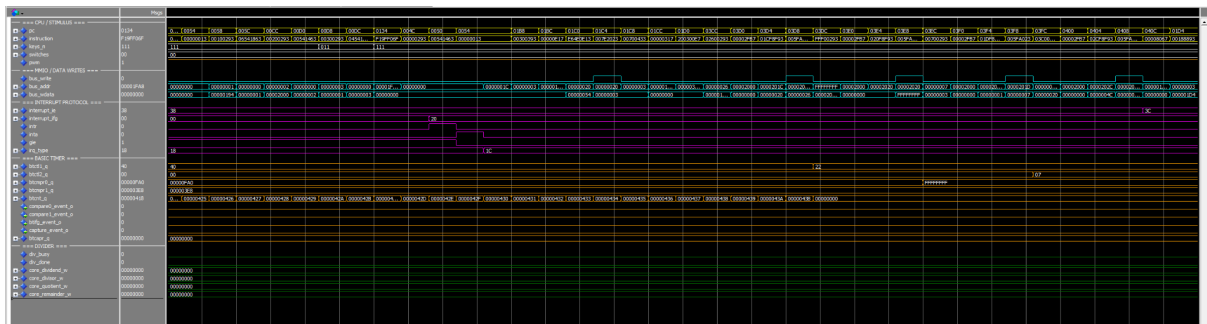


איור 9 – סימולציית ModelSim של Test3 – תכנות מחדש של תקופת ה-Basic Timer.

Test4 – PWM / Output Compare 5.4

החלק הראשון של Test4 בודק את מצב ה-Output Compare של ה-Basic Timer ואת יצירת אות ה-PWM. לאחר לחיצה על KEY2 מבוצעות כתיבות MMIO לרגיסטרי הטיימר, ובפרט ל-BTCMPR0 ול-BTCMPR1.

BTCMPR0 קובע את תקופת ה-PWM ונשאר בערך 0x00000FA0 (4000 מחזורי SMCLK), בעוד BTCMPR1 קובע את נקודת ההשוואה בתוך המחזור. ה-Benchmark משנה את ערך BTCMPR1 כך שמתקבלים Duty Cycles של 50%, 75%, 87.5% ו-93.75%. בתרשים ModelSim ניתן לראות את כתיבות ה-MMIO ואת שינוי ערכי רגיסטרי ההשוואה; ההתנהגות הפיזית של אות ה-PWM עצמו נבדקת בהמשך באמצעות SignalTap.

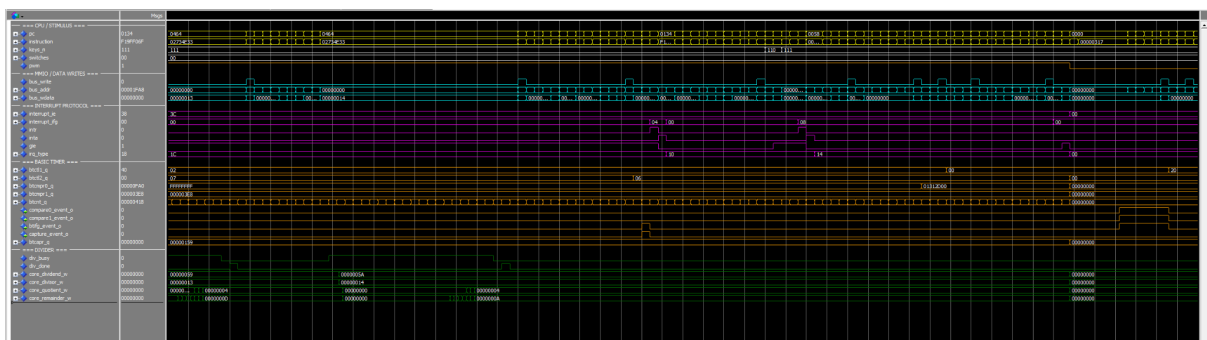


איור 10 – סימולציית ModelSim של Test4 – תכנות ה־Basic Timer במצב Output Compare / PWM.

Test4 – Input Capture and Divider 5.5

בחלק נוסף של Test4 נבדק מצב ה־Input Capture של ה־Basic Timer, המשמש למדידת זמן הריצה של פעולות DIV ו־REM. מאיץ החלוקה פועל במשך מספר מחזורי DIVCLK, ובסיום הפעולה מופק אירוע המשמש ללכידת ערך BTCNT לתוך BTCAPR.

בתרשים ניתן לראות את אותות ה־busy/done של מאיץ החלוקה, את האופרנדים והתוצאות, ובמקביל את אירוע ה־Capture ואת עדכון BTCAPR. אפליקציית ה־Benchmark מבצעת סדרת פעולות REM ולאחר מכן DIV, וה־Testbench מאמת שתוצאות המערכים נשמרו ושערכי זמן הריצה שנלכדו אינם אפס. בדיקה זו מאמת את העבודה המשותפת בין שלושה חלקים מרכזיים במערכת: ליבת המעבד, מאיץ ה־Divider וה־Basic Timer.



איור 11 – סימולציית ModelSim של Test4 – Input Capture ומדידת פעולת ה־Divider.

ארבע בדיקות ה־Benchmark מאפשרות לבדוק את המערכת ברמת אינטגרציה מלאה ולא רק ברמת יחידות בודדות. הסימולציות מאמתות את פעולת ה־GPIO והלחצנים, מנגנון הפסיקות, ה־Basic Timer במצב Input Capture ו־Compare, Output Compare, וכן את פעולת מאיץ ה־DIV/REM והתקשורת שלו עם ליבת המעבד.

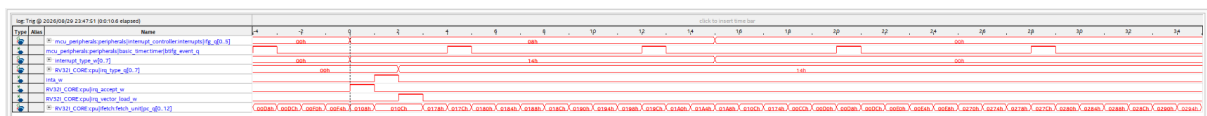
6. אימות חומרתי באמצעות SignalTap

לאחר השלמת האימות הפונקציונלי ב־ModelSim, בוצע אימות של המערכת על גבי כרטיס ה־FPGA באמצעות SignalTap. מטרת הבדיקה הייתה לוודא שהתנהגות המערכת שנצפתה בסימולציה מתקיימת גם במימוש הפיזי לאחר סינתזה ו־Place & Route. במהלך הרצת אפליקציות ה־Benchmark נלכדו אותות פנימיים מרכזיים, ובהם רגיסטר ה־IFG, ערך ה־TYPE, אותות INTA, irq_accept_w ו־irq_vector_load_w, רצף ה־PC, רגיסטרי ה־Basic Timer ואותות מאיץ החלוקה.

Test1 – KEY1 Interrupt 6.1

במהלך הרצת Test1 נלכד ב־SignalTap תהליך הטיפול בפסיקת KEY1. באיור 12 ניתן לראות כי רגיסטר ה־IFG מקבל את הערך 08h, המתאים ל־KEY1IFG, ובמקביל interrupt_type_w מקבל את הערך 14h, בהתאם לטבלת וקטורי הפסיקות. לאחר מכן הערך 14h נלכד ב־irq_type_q.

בהמשך הלכידה מופעלים irq_accept_w ו־INTA, irq_vector_load_w, ובמקביל משתנה רצף ה־PC. בכך מאומת בחומרה מסלול הטיפול בפסיקת KEY1, החל מזיהוי מקור הפסיקה ועד להעברת המעבד לשגרת הפסיקה.

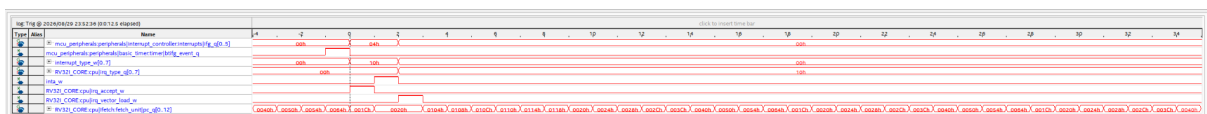


איור 12 – אימות SignalTap של Test1 – טיפול בפסיקת KEY1.

Test2 – Basic Timer Interrupt 6.2

ב־Test2 נלכדה פסיקה שמקורה ב־Basic Timer. באיור 13 ניתן לראות את עליית `btifg_event_q`, ולאחריה רגיסטר ה־IFG מקבל את הערך `04h`, המתאים ל־BTIFG. במקביל בקר הפסיקות מפיק `TYPE=10h`, והערך נלכד ב־`irq_type_q`.

לאחר זיהוי הפסיקה מופעלים `irq_accept_w` ו־`INTA` ו־`irq_vector_load_w`, ורצף ה־PC משתנה בהתאם לכניסה לשגרת הטיפול בפסיקה. הלכידה מאמתת את שרשרת הטיפול המלאה בפסיקת Basic Timer על גבי ה־FPGA.

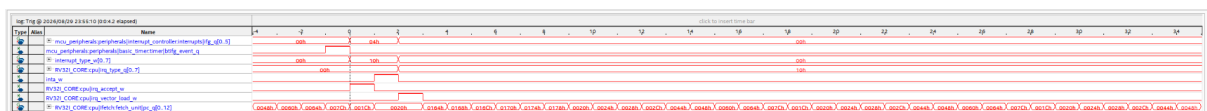


איור 13 – אימות SignalTap של Test2 – טיפול בפסיקת Basic Timer.

Test3 – Basic Timer Interrupt 6.3

במהלך Test3 נלכדה שוב פסיקה שמקורה ב־Basic Timer. באיור 14 ניתן לראות את `btifg_event_q` את עדכון ה־IFG ל־04h ואת יצירת `TYPE=10h`. לאחר מכן מופעלים אותות הטיפול בפסיקה `irq_accept_w` ו־`INTA` ו־`irq_vector_load_w`, וה־PC עובר לרצף המתאים לשגרת הפסיקה.

לכידה זו מאמתת כי גם במהלך Test3 מנגנון הפסיקות של ה־Basic Timer ממשיך לפעול באופן תקין לאחר תכנות הטיימר על ידי התוכנה.



איור 14 – אימות SignalTap של Test3 – טיפול בפסיקת Basic Timer.

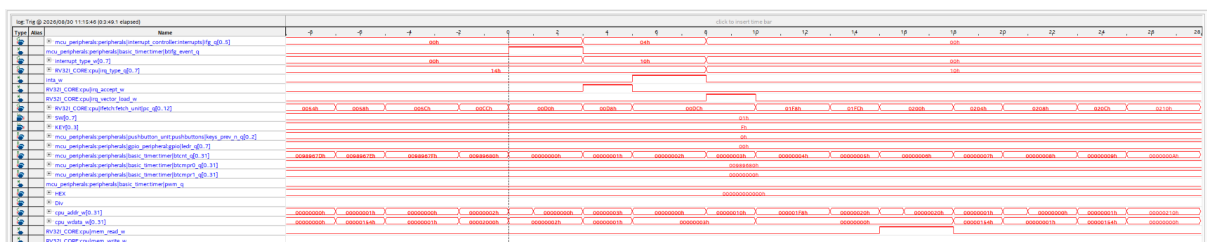
Test4 – Integrated Hardware Validation 6.4

ב־Test4 בוצע אימות חומרתי מפורט יותר של מספר מנגנונים במערכת: פסיקת Basic Timer ותגובת המעבד, פסיקת KEY1, פעולת מאיץ החלוקה ויצירת אות PWM באמצעות Output Compare.

Basic Timer Interrupt and CPU Response 6.4.1

באיור 15 ניתן לראות את BTCNT מתקדם עד לערך ההשוואה $BTCMPRO=0x00989680$. עם הגעת המונה לערך ההשוואה נוצר אירוע `btifg_event_q`, רגיסטר ה־IFG מתעדכן ל־04h ובקר הפסיקות מפיק $TYPE=10h$. לאחר מכן הערך $10h$ נלכד ב־`irq_type_q`, ומופעלים `irq_accept_w` ו־`INTA` ו־`irq_vector_load_w`.

באותה לכידה ניתן לראות גם את שינוי רצף ה־PC ואת חזרת BTCNT לערך אפס לאחר אירוע ההשוואה. בכך מאומת בחומרה הקשר המלא בין פעולת הטיימר לבין מנגנון הפסיקות ותגובת ה־CPU.



איור 15 – אימות SignalTap של Test4 – פסיקת Basic Timer ותגובת ה־CPU.

KEY1 Interrupt 6.4.2

באיור 16 מוצגת פסיקת KEY1 במהלך Test4. ניתן לראות את אוטובוס $KEY[3:0]$ משתנה מ־Dh ל־Fh, כלומר KEY1 עובר ממצב לחוץ למצב משוחרר. לאחר האירוע רגיסטר ה־IFG מקבל את הערך $08h$, ו־`interrupt_type_w` מקבל $14h$.

בהמשך $TYPE=14h$ נלכד ב־`irq_type_q`, ומופעלים `irq_accept_w` ו־`INTA` ו־`irq_vector_load_w`. במקביל משתנה ה־PC לשגרת הפסיקה. הלכידה מאמתת בחומרה את זיהוי אירוע KEY1 ואת הטיפול בו על ידי בקר הפסיקות והמעבד.



איור 16 – אימות SignalTap של Test4 – פסיקת KEY1.

Divider Accelerator 6.4.3

באיור 17 מוצגת פעולת מאיץ החלוקה במהלך Test4. ניתן לראות את התקדמות count_q עד 1Fh, ובמקביל את השינוי ההדרגתי ברגיסטרים הפנימיים של אלגוריתם החלוקה. האופרנד divisor_q שווה ל-8, ובסיום הפעולה מתקבלים quotient_q=7 ו-remainder_q=4.

בסיום החישוב מופעל done_q, ולאחר מכן מתקבל גם div_done_w בצד המעבד. הלכידה מאמתת הן את פעולת האלגוריתם הרב-מחזורי לאורך מחזורי DIVCLK והן את העברת סימון סיום החישוב והתוצאה חזרה לליבת המעבד.



איור 17 – אימות SignalTap של Test4 – פעולת מאיץ ה-Divider.

PWM – Output Compare 6.4.4

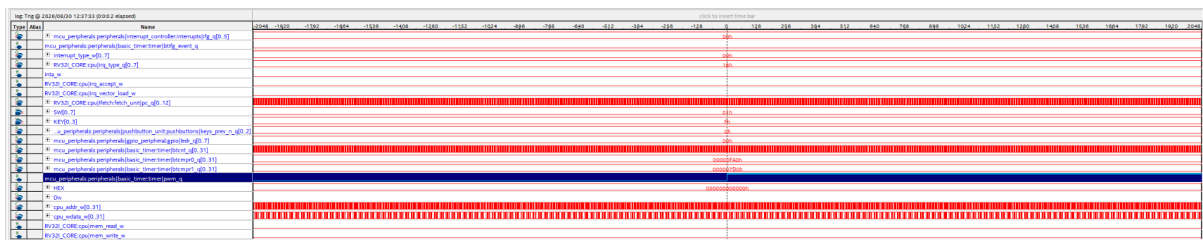
בחלק האחרון של Test4 נבדקה פעולת ה-PWM של ה-Basic Timer. בכל ארבע הלכידות BTCMPR0 נשאר קבוע בערך $0x00000FA0$, כלומר 4000 מחזורי SMCLK, בעוד BTCMPR1 משתנה וקובע את נקודת ה-Output Compare.

באיורים 18–21 מתקבלים הערכים:

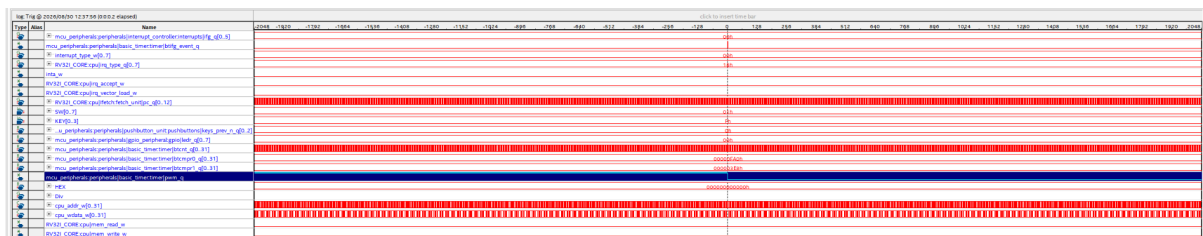
BTCMPR1= $0x000007D0$ עבור 50%, $0x00000BB8$ עבור 75%, $0x00000DAC$ עבור 87.5% ו- $0x00000EA6$ עבור 93.75%. ערכים אלו הם בהתאמה 2000, 3000, 3500 ו-3750 מתוך תקופה של 4000 ספירות. במקביל מוצג האות q_pwm, ונקודת שינוי המצב שלו משתנה בהתאם לערך ההשוואה שנבחר.

כך מאומת כי שינוי BTCMPR1 באמצעות Test4 משנה בפועל את ה-Duty Cycle של אות ה-PWM, בעוד תקופת ה-PWM נשארת קבועה. עבור $SMCLK=20\text{ MHz}$ ותקופה של 4000 מחזורים מתקבל תדר PWM של 5 kHz.

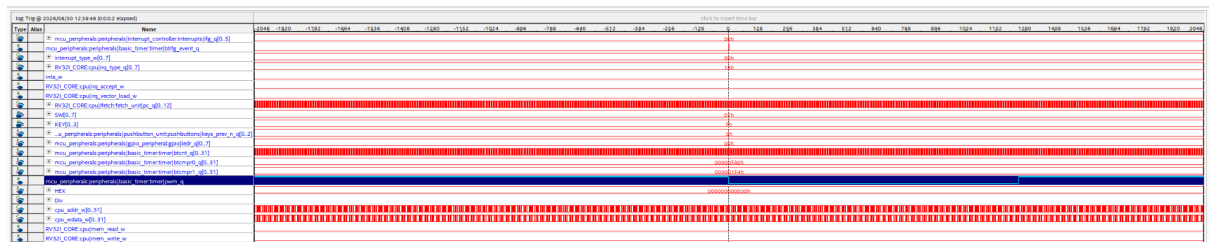
PWM Duty Cycle variation using KEY2



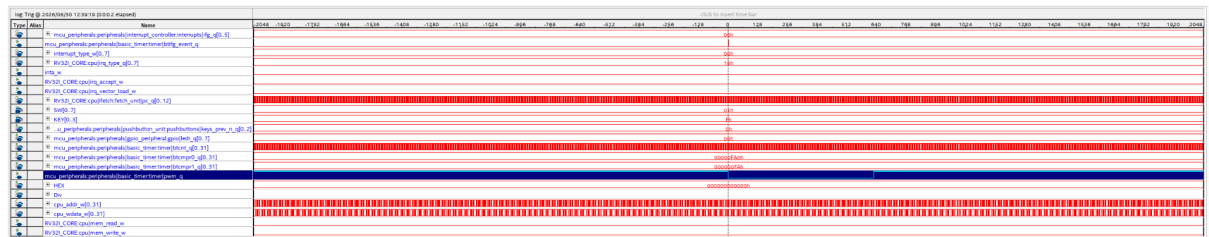
איור 18 – אימות SignalTap של PWM ב-Duty Cycle של 50%, $BTCMPR1=0x000007D0$.



איור 19 – אימות SignalTap של PWM ב-Duty Cycle של 75%, $BTCMPR1=0x00000BB8$.



איור 20 – אימות SignalTap של PWM ב־Duty Cycle של 87.5%,
 .BTCMPR1=0x00000DAC



איור 21 – אימות SignalTap של PWM ב־Duty Cycle של 93.75%,
 .BTCMPR1=0x00000EA6

האימות באמצעות SignalTap הראה כי המנגנונים המרכזיים שנבדקו ב־ModelSim פועלים גם במימוש הפיזי על גבי ה־FPGA. נצפו בפועל מסלולי הטיפול בפסיקות של KEY1 ושל ה־Basic Timer, פעולת מאיץ החלוקה הרב־מחזורי ויצירת PWM עם ארבעה ערכי Duty Cycle שונים. בכך התקבל אימות חומרתי של האינטגרציה בין ליבת המעבד, בקר הפסיקות, ה־Basic Timer ומאיץ החלוקה.

7. אפיון ה-PPA (ביצועים, הספק, שטח) של המערכת:

לאחר השלמת האימות הפונקציונלי והחומרתי בוצע אפיון PPA ב־Quartus עבור שתי תצורות של המערכת: MCU הכולל רכיבי GPIO בלבד, ו־MCU מלא הכולל גם את הרכיבים בעלי יכולת הפסיקה. האפיון כולל את ניצול משאבי ה־FPGA, תדר העבודה המקסימלי והנתיב הקריטי, וכן את צריכת ההספק של שתי התצורות. בהתאם לדרישות הפרויקט, עבור כל אחד מהמדדים מצורפים גם דוחות ה־Quartus המתאימים.

Area 7.1 - ניצול משאבי FPGA

תוצאות ה־Area מראות כי גרסת ה־GPIO בלבד משתמשת ב־1793 Logic Elements וב־1767 רגיסטרים, בעוד המימוש המלא משתמש ב־2074 Logic Elements וב־2007 רגיסטרים. כלומר, הוספת הרכיבים בעלי יכולת הפסיקה הגדילה את השימוש בלוגיקה ב־281 אלמנטים ואת מספר הרגיסטרים ב־240.

לעומת זאת, מספר פני ה־I/O נשאר 67 בשתי התצורות, כמות הזיכרון המשובץ נשארה 131,072 ביטים, ומספר המכפלים וה־PLL נשאר ללא שינוי. בשני המימושים Quartus משתמש בארבעה בלוקי DSP עבור ארבעת המכפלים מסוג Fixed Point Unsigned Multiplier. מכאן ניתן לראות שהרחבת המערכת מתבטאת בעיקר בתוספת לוגיקה ורשמים עבור הרכיבים הפריפריאליים ומנגנון הפסיקות.

Area						
	Logic elements	Registers	I/O pins	Embedded memory bits	Embedded 9-bit Multipliers	PLLs
MCU with GPIO	1793	1767	67	131,072	4	1
MCU with GPIO and Interrupt Capability	2074	2007	67	131,072	4	1

MCU with GPIO:

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sat Aug 29 19:22:31 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	RV32IMscMCU
Top-level Entity Name	RV32IMscMCU
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	1,793 / 41,910 (4 %)
Total registers	1767
Total pins	67 / 499 (13 %)
Total virtual pins	0
Total block memory bits	131,072 / 5,662,720 (2 %)
Total DSP Blocks	4 / 112 (4 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	1 / 15 (7 %)
Total DLLs	0 / 4 (0 %)

MCU with GPIO and Interrupt Capability:

Flow Summary	
<<Filter>>	
Flow Status	Successful - Sat Aug 29 17:29:23 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	RV32IMscMCU
Top-level Entity Name	RV32IMscMCU
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	2,074 / 41,910 (5 %)
Total registers	2007
Total pins	67 / 499 (13 %)
Total virtual pins	0
Total block memory bits	131,072 / 5,662,720 (2 %)
Total DSP Blocks	4 / 112 (4 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	1 / 15 (7 %)
Total DLLs	0 / 4 (0 %)

בנוסף ניתן לראות כי שני המימושים משתמשים ב-4 מכפלים מסוג Fixed Point Unsigned Multiplier, הממומשים באמצעות 4 DSP blocks.

Analysis & Synthesis DSP Block Usage Summary		
<<Filter>>		
	Statistic	Number Used
1	Independent 9x9	4
2	Total number of DSP blocks	4
3		
4	Fixed Point Unsigned Multiplier	4

Analysis & Synthesis DSP Block Usage Summary		
<<Filter>>		
	Statistic	Number Used
1	Independent 9x9	4
2	Total number of DSP blocks	4
3		
4	Fixed Point Unsigned Multiplier	4

איור 22 - צילומי מסך מ Quartus עבור Area של המערכת

Performance 7.2 - תדר מקסימלי והנתיב הקריטי

המימוש הסופי עבר Full Compilation ב־Quartus Prime 21.1 באמצעות Fitter Seed = 2. עבור גרסת ה־MCU עם GPIO בלבד התקבל תדר מקסימלי של 21.9 MHz, בעוד עבור המימוש המלא התקבל $f_{max} = 20.73$ MHz. בשתי התצורות תדר העבודה שנבחר עבור MCLK הוא 20 MHz, ולכן שתי התצורות עומדות בתדר העבודה הנדרש. במימוש המלא נשאר מרווח של 0.73 MHz בין תדר העבודה לבין ה־ f_{max} שהתקבל.

הנתיב הקריטי נמצא באזור ליבת ה־RV32I החד־מחזורית. בצילומי הנתיב הקריטי ניתן לראות מסלול ארוך העובר דרך מספר רב של רכיבי לוגיקה לאורך נתיב עיבוד הפקודה ומגיע לכיוון זיכרון הנתונים. מאחר שמדובר בארכיטקטורת Single-Cycle, הבאת הפקודה, הפענוח, בחירת האופרנדים, הביצוע וחישוב הגישה לזיכרון צריכים להסתיים כולם בתוך מחזור שעות יחיד. כתוצאה מכך מתקבל מסלול קומבינטורי ארוך, והוא שמגביל את תדר ה־MCLK המקסימלי.

Performance			
	f_{max}	Critical path (what is the slowest submodule and why does it cause the critical path)	$f_{MCLK}(= f_{sysclk})$
MCU with GPIO	21.9 MHz	הרכיב האיטי ביותר הוא ליבת ה־RV32I. הנתיב הקריטי עובר מזיכרון הפקודות ב־IFETCH אל זיכרון הנתונים DTCM. מאחר שהמעבד חד־מחזורי, שלבי הבאת הפקודה, הפענוח, הביצוע וחישוב הגישה לזיכרון חייבים להסתיים באותו מחזור שעות, ולכן נוצר מסלול קומבינטורי ארוך שמגביל את תדר ה־MCLK המקסימלי.	20 MHz
MCU with GPIO and Interrupt Capability	20.73 MHz	המסלול הקריטי נמצא בתוך ליבת ה־RV32I החד־מחזורית, ועובר בקירוב מ־IFETCH/ITCM לזיכרון ה־DTCM. הסיבה שהוא קריטי היא שבמעבד ה־Single-Cycle כל שלבי עיבוד הפקודה - הבאת הפקודה, פענוח, בחירת האופרנדים, ביצוע וחישוב הכתובת לזיכרון - חייבים להסתיים בתוך מחזור שעות אחד. לכן נוצר מסלול קומבינטורי ארוך שעובר דרך הרבה לוגיקה ו־MUXים, והוא זה שמגביל את תדר ה־MCLK המקסימלי.	20 MHz

MCU with GPIO:

Slow 1100mV 85C Model Fmax Summary

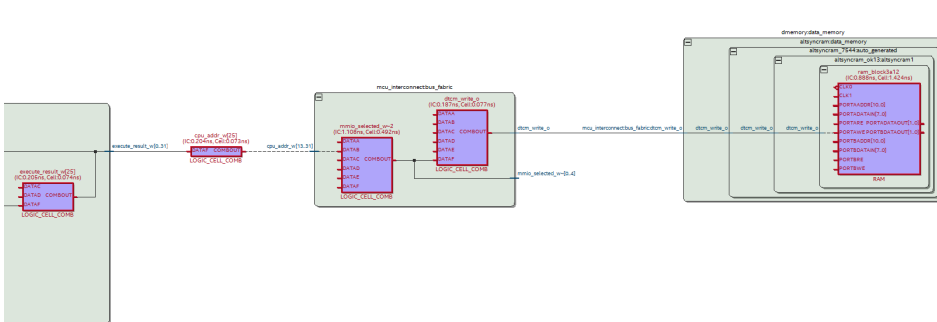
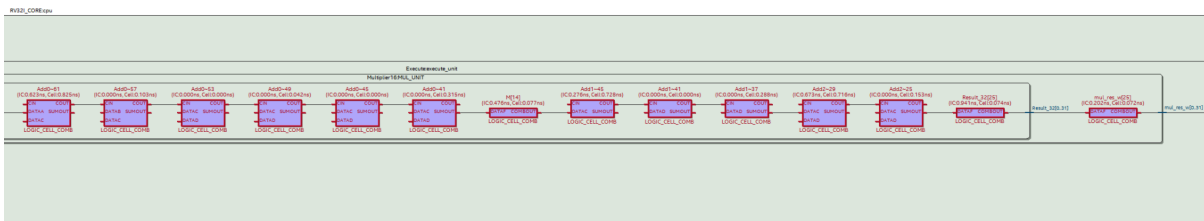
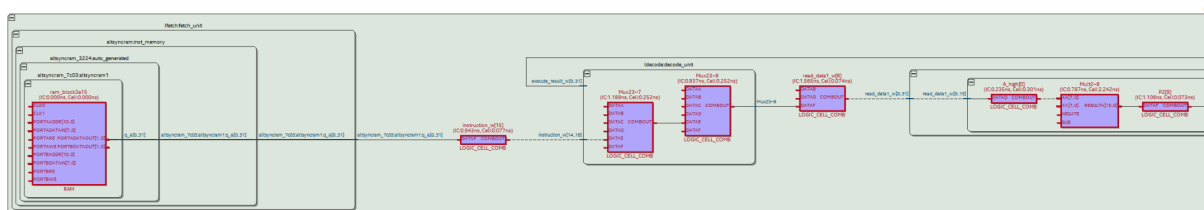
<<Filter>>

	Fmax	Restricted Fmax	Clock Name	Note
1	21.9 MHz	21.9 MHz	clock_pll altpll_component auto_generated generic_pll1~PLL_OUTPUT_COUNTER divclk	
2	68.51 MHz	68.51 MHz	altera_reserved_tck	
3	96.4 MHz	96.4 MHz	CLOCK_50	

Full critical path:



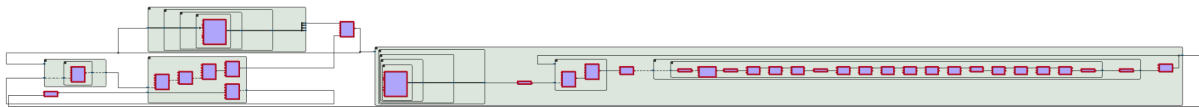
zoom in (3 parts):



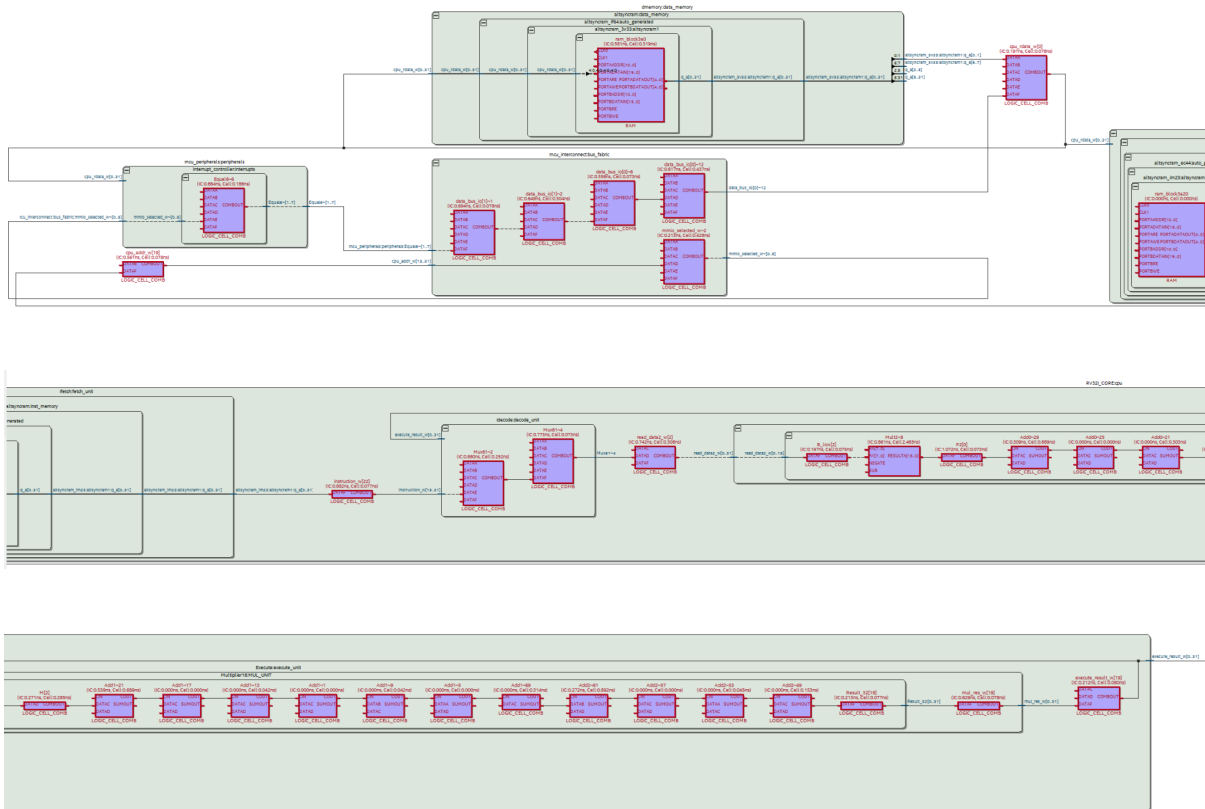
MCU with GPIO and Interrupt Capability:

Slow 1100mV 85C Model Fmax Summary				
	Fmax	Restricted Fmax	Clock Name	Note
1	20.73 MHz	20.73 MHz	mclk_pll altpll_component auto_generated generic_pll1~PLL_OUTPUT_COUNTER divclk	
2	96.83 MHz	96.83 MHz	altera_reserved_tck	
3	105.65 MHz	105.65 MHz	divclk_pll altpll_component auto_generated generic_pll1~PLL_OUTPUT_COUNTER divclk	

Full critical path:



zoom in (3 parts):



איור 23 - צילומי מסך מ Quartus עבור Performance של המערכת

Power 7.3 - צריכת הספק

עבור גרסת ה־MCU עם GPIO בלבד התקבלה צריכת הספק כוללת של 461.52 mW, המורכבת מ־415.39 mW הספק סטטי, 31.07 mW הספק דינמי ו־15.05 mW עבור I/O. עבור המימוש המלא התקבלה צריכה כוללת של 456.13 mW, מתוכה 415.36 mW סטטי, 29.00 mW דינמי ו־11.77 mW עבור I/O.

בשתי התצורות ניתן לראות כי עיקר צריכת ההספק היא סטטית, והיא כמעט זהה בין המימושים. ההבדל בצריכת ההספק הכוללת קטן יחסית - כ־5.39 mW - כאשר בדוח שהתקבל למימוש המלא הערך הכולל אף נמוך מעט מזה של גרסת ה־GPIO בלבד. מאחר ש־Quartus מציין בדוחות אלו Power Estimation Confidence: Low, יש להתייחס לערכי ההספק כאומדן התלוי גם בפעילות המיתוג ששימשה את ה־Power Analyzer, ולא להסיק מההפרש הקטן לבדו שמימוש אחד בהכרח חסכוני יותר מהשני.

Power				
	Total power consumption	Static power consumption	Dynamic power consumption	I/O power consumption
MCU with GPIO	461.52 mW	415.39 mW	31.07 mW	15.05 mW
MCU with GPIO and Interrupt Capability	456.13 mW	415.36 mW	29.00 mW	11.77 mW

MCU with GPIO:

Power Analyzer Summary	
<<Filter>>	
Power Analyzer Status	Successful - Sat Aug 29 20:52:09 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	RV32IMscMCU
Top-level Entity Name	RV32IMscMCU
Family	Cyclone V
Device	5CSXFC6D6F31C6
Power Models	Final
Total Thermal Power Dissipation	461.52 mW
Core Dynamic Thermal Power Dissipation	31.07 mW
Core Static Thermal Power Dissipation	415.39 mW
I/O Thermal Power Dissipation	15.05 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

MCU with GPIO and Interrupt Capability:

Power Analyzer Summary	
<<Filter>>	
Power Analyzer Status	Successful - Sat Aug 29 20:41:34 2026
Quartus Prime Version	21.1.0 Build 842 10/21/2021 SJ Lite Edition
Revision Name	RV32IMscMCU
Top-level Entity Name	RV32IMscMCU
Family	Cyclone V
Device	5CSXFC6D6F31C6
Power Models	Final
Total Thermal Power Dissipation	456.13 mW
Core Dynamic Thermal Power Dissipation	29.00 mW
Core Static Thermal Power Dissipation	415.36 mW
I/O Thermal Power Dissipation	11.77 mW
Power Estimation Confidence	Low: user provided insufficient toggle rate data

איור 24 - צילומי מסך מ Quartus עבור Power של המערכת

בסיכום אפיון ה־PPA, הוספת ה־Basic Timer, בקר הפסיקות ושאר הרכיבים בעלי יכולת הפסיקה הגדילה בעיקר את השימוש בלוגיקה וברגיסטרים והורידה במידה מסוימת את ה־fmax. עם זאת, המימוש המלא עדיין עומד בתדר העבודה הנדרש של 20 MHz, ללא שינוי בכמות הזיכרון המשובץ, במספר מכפלי ה־DSP או במספר ה־PLL-ים. צריכת ההספק הכוללת נשארה באותו סדר גודל בשתי התצורות.

8. מסקנות

בפרויקט תוכננה ומומשה מערכת MCU המבוססת על ליבת RV32IM Single-Cycle, הכוללת זיכרונות ITCM ו-DTCM, ממשקי GPIO בשיטת Memory Mapped I/O, מערכת פסיקות, Basic Timer ומאיץ חלוקה רב-מחזורי עבור פעולות DIV ו-REM. במהלך העבודה בוצעה אינטגרציה בין כלל הרכיבים וכן התאמה בין מספר תחומי שיעון באמצעות מנגנוני CDC.

תקינות המערכת נבדקה תחילה בסימולציות ModelSim באמצעות ארבע אפליקציות ה-Benchmark, ולאחר מכן נבדקה גם על גבי ה-FPGA באמצעות SignalTap. הבדיקות אימתו בין היתר את הטיפול בפסיקות KEY ובפסיקות ה-Basic Timer, את פעולת מאיץ החלוקה, את מצבי ה-Output Compare וה-Input Capture ואת יצירת אות ה-PWM במספר ערכי Duty Cycle. בכך התקבל אימות הן ברמת הסימולציה והן במימוש הפיזי של האינטגרציה בין ליבת המעבד והרכיבים הפריפריאליים.

מניתוח ה-PPA התקבל כי המימוש המלא משתמש ב-2074 Logic Elements וב-2007 רגיסטרים, ומגיע ל- $f_{max} = 20.73 \text{ MHz}$. מאחר שתדר העבודה שנבחר עבור MCLK הוא 20 MHz, המימוש עומד בדרישת התזמון. הוספת הרכיבים בעלי יכולת הפסיקה הגדילה בעיקר את השימוש בלוגיקה וברגיסטרים והקטינה מעט את ה- f_{max} ביחס לגרסת ה-GPIO בלבד, אך ללא שינוי בכמות הזיכרון המשובץ, במספר בלוקי ה-DSP או במספר ה-PLL-ים. צריכת ההספק הכוללת של המימוש המלא הוערכה ב-456.13 mW.

בסיום הפרויקט התקבלה מערכת MCU מלאה העומדת בדרישות הפונקציונליות של הפרויקט ומדגימה שילוב בין ליבת מעבד, רכיבי Memory Mapped I/O, מנגנון פסיקות, טיימר ומאיץ חומרה במימוש יחיד על גבי FPGA.